Auditing Speedup Claims for GPU Kernels from Language Models

Aviad Cohen
Independent Researcher

Abstract

Systems that generate GPU kernels with language models often time several candidates and promote the fastest. This couples search to evaluation. Timing noise and evaluator state can then be mistaken for performance. We present OpenKernelForge, an evaluation protocol that freezes candidates before timing and confirms the selected winner in fresh processes. It combines persistent modules, contract checks, randomized paired CUDA events, formal controls, and checksummed artifacts. We apply the protocol to 48 KernelBench L1 tasks selected without performance data. Of 141 evaluated Gemini candidates, 27 pass every gate and cover 10 tasks. No selected task winner clears a prespecified 2% margin over eager execution in screening or confirmation. One candidate confirms a $2.001\times$ gain over `torch.compile` while remaining below eager. A deterministic control with 80 easy candidates yields no false promotions. In a calibrated stress test near parity, screening promotes three of four task winners at $K = 8$, but only two confirm. The interval across four tasks includes zero, so this is evidence for a mechanism rather than a population estimate. A lifecycle ablation finds $1.053\times$ median host latency inflation without a corresponding CUDA event change. Independent confirmation can therefore change the claim supported by a kernel search.

1 Introduction

Systems that synthesize GPU kernels may produce many implementations for one tensor program. They commonly time each candidate and retain the fastest. The same latency observation then serves as both a search signal and evidence for the chosen kernel. Larger candidate pools increase the chance of selecting a favorable fluctuation. Reusing that observation for the final claim resembles test set overfitting.

The evaluator creates a second source of risk. Incorrect state initialization can invalidate the comparison. Constructing a reference model inside the timed region can inflate its latency. A permissive fallback can bypass the intended kernel altogether. Each error can produce an apparent speedup without better GPU code.

KernelBench and recent extensions evaluate correctness and speed against practical baselines [Ouyang et al., 2025, Zhang et al., 2026, Lange et al., 2025]. Harness research also stresses constrained execution and artifact preservation [Shui et al., 2026]. Systems measurement work has shown that small experimental choices can reverse a performance conclusion [Mytkowicz et al., 2009, Curtsinger and Berger, 2013]. We connect these lines of work through a promotion protocol for generated kernels. One set of data screens candidates. Fresh processes then confirm the selected winner.

We ask three questions. **RQ1: repeatability.** Under a fixed budget of three candidates, how many valid KernelBench tasks produce a screening win and an independently confirmed win? **RQ2: selection bias.** How does the gap between screening and confirmation change with candidate budget in an easy control and a calibrated parity stress test? **RQ3: evaluator validity.** How much apparent speedup arises when the reference model is rebuilt and moved to the GPU inside the measured call?

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

OpenKernelForge contributes an evaluation design. It uses persistent modules with frozen provenance. Randomized paired blocks support screening, while fresh processes provide confirmation. Null, slowdown, and lifecycle controls must pass before performance claims are enabled. A checksum ledger preserves the evidence needed to audit both the candidate and the evaluator.

The claim ladder is deliberately strict:

$$\text{contract-valid and correct} \;\rightarrow\; \text{beats compiler} \;\rightarrow\; \text{beats eager} \;\rightarrow\; \text{independently confirmed.}$$

Each arrow is a separate empirical claim.

Recent verifier studies show that one numerical comparison is not enough to establish kernel correctness [Shah and Shrestha, 2026, Li et al., 2026]. Our protocol therefore treats output contracts, numerical correctness, lifecycle validity, and speed as distinct gates. The historical fused8 example is only a motivation. The reported campaign passed its control gates and is derived from checksummed records.

2 Evaluation protocol

Task lifecycle and correctness. Official KernelBench tasks define a reference `Model` and expect a persistent `ModelNew` [Ouyang et al., 2025]. We instantiate both from the same frozen constructor arguments. We restore the random state and move both modules to the GPU before warmup. Source, initialization, and state hashes record provenance. Candidates face static policy checks and five seeded input cases. Verification covers repeated execution, exact output structure, aliasing, input effects, special values, shape, dtype, and official tolerances. Before timing, a `TorchDispatchMode` audit rejects high level ATen compute and requires an observed Triton launch [Tillet et al., 2019]. These checks enforce the task contract. They are not a security sandbox.

Task selection. The study uses official KernelBench L1 commit `423217d9`. Before candidate generation, a memory preflight selected 48 tasks. Selection used a deterministic round robin across families and lexical order within each family. The cap was 8 GiB of known resident memory. We amended the original target of 50 before the manifest was frozen. Only 49 of 100 tasks met the cap, and the next memory tier began at 20 GiB. One feasible task was reserved for a shakedown that did not enter the results. No candidate or timing field informed this amendment.

Paired screening and fresh confirmation. For task t, candidate c, process r, and paired block b, define

$$z_{tcrb} = \log(\widetilde{T}^{\text{eager}}_{trb} / \widetilde{T}^{\text{c}}_{tcrb}), \tag{1}$$

where each $\widetilde{T}$ is a per launch CUDA event latency on the same input snapshot [NVIDIA, 2026]. One process screens three candidates per task with 20 randomized paired blocks. We freeze the candidate with the largest median z. Tasks with no valid candidate remain explicit INVALID rows. The selected winner then receives 20 blocks in each of seven fresh processes. Each process uses new seeds. Four processes run in the first wave and three in the second, with at least 30 minutes between waves on the same GPU and software stack. Every interval adapts its launch count to cover at least 1.5 ms. Method order is randomized. A read and write buffer sized from the reported L2 capacity perturbs cache state in the same way for both methods. Compile cost is recorded separately from runtime [Ansel et al., 2024]. Promotion is measured against eager execution.

The practical margin is $\delta = 0.02$. We fixed this default before timing to require a meaningful gain over parity. It was not calibrated to the observed T4 noise. Primary outcomes are valid tasks, screening wins, confirmed wins, false promotions, and median screening to confirmation optimism with a task bootstrap interval. A secondary process analysis uses a one sided lower bound, a centered bootstrap test, and Benjamini–Hochberg correction. The executable protocol fixes the seeds, resampling rules, telemetry, precision, and failure behavior. The appendix summarizes these choices.

Controls and candidate multiplicity. Seven process calibrations compare eager execution with two controls. The first is an identical wrapper. The second adds a known amount of GPU work. A separate lifecycle study uses 24 processes to measure reference reconstruction and transfer with both synchronized host latency and an enclosing CUDA event. Performance claims are disabled when a required control is absent or fails.

Table 1: Campaign summary. No KernelBench winner clears eager. Confirmation removes one of three promotions in the parity stress test.

Study	Evidence	Screen	Confirm	Dropped	Key estimate
RQ1 holdout	10 valid / 48	0	0	–	1.014 ratio $[0.915, 1.170]$
RQ2 easy, $K = 20$	4 tasks	1.00	1.00	0	0.0000 log optimism
RQ2 parity, $K = 8$	4 tasks	0.75	0.50	1/3	0.0072 log optimism
RQ3 lifecycle	24 rows	–	–	–	1.053 host $[0.998, 1.114]$

RQ2 uses two deterministic fused8 regimes. The easy regime freezes and confirms 20 candidates per task for $K \leq 20$. The parity stress test also freezes 20 variants per task. Each variant preserves the base output and adds calibrated Triton copy work into unused scratch. Fractional work units copy a prefix of the first input. Three calibration processes select eight variants per task inside the declared $[0.98, 1.04]$ window. Calibration data do not enter the estimates. All 32 selected candidates receive screening and seven confirmation processes. Confirmation starts at least 30 minutes after screening. We use 4,000 resamples to estimate budgets $K \in \{1, 2, 3, 5, 8\}$. The two regimes use different artifacts and GPUs. Neither is inferred from the KernelBench candidate pool.

3 Results

Motivating example. A deterministic `bias_relu` template measured 1.029× above eager in one fused8 screening run, then 0.976× at repeated median. Model sampling cannot explain the reversal. We use it as an example, not an estimate of how often such reversals occur. The new campaign assigns a distinct evidence path to each research question (Table 1 and Figure 1).

RQ1: validity precedes speed. We froze all 144 Gemini candidates before timing. Of the 141 records that reached evaluation, 77 failed static policy and 9 failed correctness. Another 28 failed during compilation or execution. The remaining 27 passed every gate. A compiler OOM prevented evaluation of three candidates from one task. The valid candidates covered 10 of 48 tasks, all matrix multiplications.

At screening, one selected winner beat `torch.compile` by 1.826× but remained below eager at 0.937×. A distinct check across seven A4500 processes confirmed a 2.001× gain over the compiler. All 10 selected winners remained below eager in screening and confirmation. The false promotion fraction is therefore undefined, not zero. The median ratio from screening to confirmation was 1.014 with task bootstrap interval $[0.915, 1.170]$. This interval contains parity and does not support directional selection optimism.

RQ2: candidate budget matters near parity. The easy regime acts as a negative control. All 80 deterministic candidates confirmed above the margin. Screening and confirmation rates were 1.0 for every $K \leq 20$ on the original T4 and in a replication on the same A4500.

The calibrated stress test placed 32 frozen candidates near parity. At $K = 8$, screening promoted three of four task winners, while confirmation retained two. The removed `bias_gelu` winner moved from 1.0271× to 1.0001×. The `bias_relu` and `rmsnorm` winners remained above the margin. Across candidate resamples, the screening and confirmation rates diverged from 0.154 versus 0.124 at $K = 1$ to 0.750 versus 0.500 at $K = 8$. Median log optimism at $K = 8$ was 0.00725. Its task bootstrap interval, $[-0.01239, 0.02661]$, contains zero. The removed winner demonstrates the mechanism but does not establish a population rate. Margin sensitivity appears in Appendix A.

RQ3: the timing boundary changes the result. The lifecycle control completed 24 process rows across eight tasks. Rebuilding and transferring the reference inside the measured call raised synchronized host latency by a median factor of 1.053. The enclosing CUDA event ratio was 1.0001. Their process row IQRs were $[0.998, 1.114]$ and $[0.9919, 1.0019]$. Reference lifecycle work can therefore alter a host latency claim without changing the enclosed device execution.

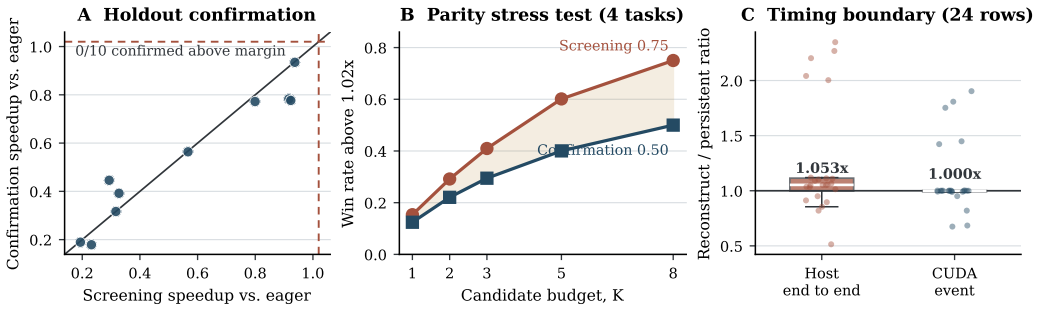

Figure 1: Independent confirmation changes the supported claim. Panel A shows that all 10 valid KernelBench winners remain below the 1.02× eager margin. Panel B isolates one screening promotion that does not confirm. Panel C compares host and CUDA event ratios for the lifecycle control.

4 Discussion

Relation to prior work. KernelBench measures kernel generation at scale [Ouyang et al., 2025]. KernelBench-Verified and robust-kbench strengthen hidden tests, precision settings, memory analysis, and verification scenarios [Zhang et al., 2026, Lange et al., 2025]. FastKernels studies alignment with production workloads [Oliaro et al., 2026]. Harness Engineering preserves workload evidence during optimization [Shui et al., 2026]. Our focus is narrower. We study whether the evidence supports promotion of one measured winner after candidate search.

Contract-Grade Verifier formalizes structural and behavioral checks [Shah and Shrestha, 2026]. Correct but Slow shows why numerical validity alone does not establish replacement quality [Li et al., 2026]. Our verifier enforces output structure, aliasing, mutation, special values, and repeat execution. Distribution and precision oracles remain task specific.

Scope. The KernelBench study uses one Tesla T4, one model, three candidates per task, and a deterministic feasible subset rather than a random sample. All valid tasks are matrix multiplications. The result therefore characterizes this campaign, not KernelBench L1 as a whole. The fused8 studies cover four tasks in one shape regime. Their stress test and hardware control use one RTX A4500. Calibration deliberately enriches candidates near parity, and the interval across four tasks is wide. Seven confirmation processes provide a practical holdout, not a model of tail behavior.

The fixed 2% margin is a practical threshold declared before timing. It was not calibrated to the observed noise. The multiplicity studies use deterministic templates, so they answer a different question from candidate generation. We do not analyze CUDA graphs, multiple GPUs, deployment cost, or model rankings. The compiler baseline exists for 47 of 48 tasks, but only one selected winner receives compiler confirmation in fresh processes.

Conclusion. Plausible source and one fast timing sample do not establish a kernel speedup. The protocol makes screening, confirmation, and evaluator validity separately auditable. In this study, the external campaign supports no gain over eager. The easy control shows a regime with no false promotion. The parity stress test removes one screening winner, and the lifecycle study isolates a host timing confound. Independent confirmation is a useful default even when the supported result is a bound or a null finding.

References

Jason Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 929–947. ACM, 2024. doi: 10.1145/3620665.3640366.

Charlie Curtsinger and Emery D. Berger. STABILIZER: Statistically sound performance evaluation. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13, pages 219–228. ACM, 2013. doi: 10.1145/2451116.2451141.

Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic CUDA kernel benchmarking, verification, and optimization, 2025. arXiv:2509.14279.

Tingxi Li, Ravishka Rathnasuriya, and Wei Yang. Correct but slow: An empirical study of the GPU kernel evaluation gap in modern domain-specific languages, 2026. arXiv:2607.04454.

Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '09, pages 265–276. ACM, 2009. doi: 10.1145/1508244.1508275.

NVIDIA. *CUDA Runtime API: Event Management*. NVIDIA, 2026. CUDA Runtime API documentation, https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__EVENT.html.

Gabriele Oliaro, Yichao Fu, May Jiang, Owen Lu, Junli Wang, Zhihao Jia, Hao Zhang, and Samyam Rajbhandari. FastKernels: Benchmarking GPU kernel generation in production, 2026. arXiv:2605.23215.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels?, 2025. arXiv:2502.10517.

Rishi Shah and Rishav Shrestha. A contract-grade verifier for LLM-generated GPU kernels, and a native Blackwell backward for the gated-linear-recurrence family, 2026. arXiv:2608.12700.

Yue Shui, Chenyu Ma, Hangfei Xu, Shengzhao Wen, and Yanpeng Wang. Harness engineering for LLM-driven GPU kernel generation, 2026. arXiv:2607.17979.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19. ACM, 2019. doi: 10.1145/3315508.3329973.

Yunxiang Zhang, Ping Yu, Jianyu Wang, Max (Xiangjun) Fan, Julian Reed, Azalia Mirhoseini, and Will Su. KernelBench-Verified: Do LLM-generated kernels actually beat PyTorch?, 2026. arXiv:2607.16241.

A Artifact and analysis details

The executable protocol is stored in `configs/workshop2026_holdout_protocol.yaml`. Before generation, task selection writes a checksummed JSON manifest and a compact CSV. Candidate manifests bind each source file to the task manifest. They preserve prompt and response hashes, model identifiers, and provider metadata when available. Screening freezes each selected winner in a second checksummed manifest before confirmation. Worker records capture policy and verification outcomes, process and environment metadata, launch counts, cache size, method order, input hashes, block latency, clocks, and power. Analysis reads only these records.

Control gate and correctness matrix. Seven fresh calibration processes compare persistent eager execution with an identical wrapper and a wrapper that adds calibrated GPU work. The gate requires a null median ratio in $[0.995, 1.005]$, no null promotion beyond 2%, and a detected slowdown in $[0.02, 0.08]$. A separate lifecycle study covers the first selected task in each family, capped at eight. Each task requires three complete processes and ten blocks per process. Timing includes synchronized host latency and an enclosing CUDA event. A failed control blocks screening.

Across the 24 process rows, host inflation had median 1.053 and IQR $[0.998, 1.114]$. CUDA event inflation had median 1.0001 and IQR $[0.9919, 1.0019]$. A task bootstrap with 20,000 samples resampled the eight tasks while retaining the three rows for each sampled task. It produced intervals $[0.950, 1.557]$ and $[0.992, 1.212]$. Their width shows task heterogeneity. The ablation identifies a timing boundary mechanism, not a population effect size.

Correctness uses seeds 1103, 2207, 3301, 4409, and 5519. Every case executes the same logical input twice. Checks cover nested structure, shape, dtype, aliases, input effects, nonfinite masks, and official tolerances. Runtime policy requires an observed Triton launch and rejects ATen compute outside the allocation allowlist. Separate oracles remain necessary for task specific distributions, layouts, and accumulation behavior.

Controlled multiplicity artifact contract. The easy RQ2 manifest freezes 20 deterministic templates for each of `bias_relu`, `residual_add_relu`, `bias_gelu`, and `rmsnorm_small`. The parity manifest freezes 20 delayed variants per task. Each variant adds a fixed number of Triton copy passes from the first input into unused scratch without changing the output. Fractional work units copy an input prefix. Three calibration processes select eight candidates per task in the $[0.98, 1.04]$ window. Calibration is excluded from the estimates. Each selected candidate receives one screening process and seven confirmation processes. Two earlier grids remain as design provenance because they did not reach screening. Reports preserve counts, rates, winners, and median selection optimism. None is inferred from the KernelBench candidate pool.

Promotion-margin sensitivity. The primary threshold remains 2%. A post hoc check considers 0%, 1%, 2%, 3%, and 5%. It changes no RQ1 conclusion because all 10 KernelBench winners remain below eager. Among the four parity stress test winners, one screening promotion fails confirmation at 1%, 2%, and 3%. All four clear parity in both stages, and none clears 5%.

Historical audit boundary. The OpenKernelForge report has 17 pages of evaluator audit and implementation details. Historical candidates and profiler rows remain for provenance but do not enter this paper's performance tables. The fused8 summary supplies only fixed observations with surviving artifacts. Aggregate summaries cannot recover the full 160 candidate screening and confirmation frequency.

Candidate funnel and compiler rung. The 144 generated candidates form mutually exclusive categories. Static policy rejects 77. Nine fail numerical or repeat correctness, and 28 fail during compilation or execution. A total of 27 pass every gate. Three do not reach evaluation because their task's compiler baseline exhausts memory. No candidate fails only an output contract. Compiler baselines complete for 47 of 48 tasks. One of the 10 selected winners clears compiler parity and the 2% margin at screening. It reaches 1.826× versus the compiler but 0.937× versus eager. A separate check across seven A4500 processes reuses the frozen source and confirms a median of 2.001× versus the compiler. Process medians range from 1.993× to 2.003×. Runtime excludes compile cost. Latencies for compilation and the first call remain preserved.

Completed artifact set. The holdout contains 48 screening tasks, 70 confirmation records, and 249 ledger entries. The easy study has four screening records, 28 confirmation processes, and 67 ledger entries. The parity study has 12 calibration records, four screening records, 28 confirmation records, and a verified ledger of 94 files. Calibration passes in seven processes with null ratio 0.999998 and detected slowdown 0.0563. The lifecycle gate passes all 24 required rows. The holdout and easy grid use 2.091 recorded T4 worker hours. The parity study uses 0.368 A4500 worker hours. Replication on the same A4500 adds 32 records and 0.424 hours while preserving 1.0 screening and confirmation rates at every budget. The compiler check adds seven records and 0.015 hours. The build rejects pending evidence, enforces four main pages, and bundles the original `neurips_2026.sty`. The reviewer release at `https://github.com/aviad12g/kernel-forge` includes source, protocols, derived tables, instructions, and checksummed records.